

# 7

## THE ACCESS CHECK PROCESS



We've covered the first two components of the SRM: the security access token and the security descriptor. Now we'll define its final component: the access check process, which accepts the token and the security descriptor and applies a fixed set of rules to determine whether an application can access a resource.

We'll start by discussing the APIs you can call to perform an access check. Then we'll take a deep dive into the implementation of the access check inside the Windows kernel, detailing how this check processes the different parts of the security descriptor and `Token` object to generate a final granted access value for the resource. In doing so, we'll develop our own basic implementation of the access check process using a PowerShell script.

### Running an Access Check

When a caller attempts to open a resource, the kernel performs an access check based on the caller's identity. The API used to run the access check depends on whether it's being called from kernel mode or user mode. Let's start by describing the kernel-mode API.

## ***Kernel-Mode Access Checks***

The `SeAccessCheck` API implements the access check process in kernel mode. It accepts the following parameters:

**Security descriptor** The security descriptor to use for the check; must contain both owner and group SIDs

**Security subject context** The primary and impersonation tokens for the caller

**Desired access** An access mask for the access requested by the caller

**Access mode** The caller's access mode, set to either `UserMode` OR `KernelMode`

**Generic mapping** The type-specific generic mapping

The API returns four values:

**Granted access** An access mask for the access the user was granted

**Access status code** An NT status code indicating the result of the access check

**Privileges** Any privileges used during the access check

**Success code** A Boolean value; if `TRUE`, the access check succeeded

If the access check succeeds, the API will set the granted access to the desired access parameter, the success code to `true`, and the access status code to `STATUS_SUCCESS`. However, if any bit in the desired access is not granted, it will set the granted access to `0`, the success code to `false`, and the access status code to `STATUS_ACCESS_DENIED`.

You might wonder why the API bothers returning the granted access value if all bits in the desired access must be granted for this value to indicate a success. The reason is that this behavior supports the `MaximumAllowed` access mask bit, which the caller can set in the desired access parameter. If the bit is set and the access check grants at least one ac-

cess, the API returns `STATUS_SUCCESS`, setting the granted access to the maximum allowed access.

The security subject context parameter is a pointer to a `SECURITY_SUBJECT_CONTEXT` structure containing the caller's primary token and any impersonation token of the caller's thread. Typically, kernel code will use the kernel API `SeCaptureSubjectContext` to initialize the structure and gather the correct tokens for the current caller. If the impersonation token is captured, it must be at Impersonation level or above; otherwise, the API will fail and the access status code will be set to `STATUS_BAD_IMPERSONATION_LEVEL`.

Note that the call to `SeAccessCheck` might not occur in the thread that made the original resource request. For example, the check might have been delegated to a background thread in the `system` process. The kernel can capture the subject context from the original thread and then pass that context to the thread that calls `SeAccessCheck`, to ensure that the access check uses the correct identity.

### The Access Mode

The access-mode parameter has two possible values, `UserMode` and `KernelMode`. If you pass `UserMode` to this parameter, all access checks will continue as normal. However, if you pass `KernelMode`, the kernel will disable all access checks. Why would you want to call `SeAccessCheck` without enforcing any security? Well, usually, you won't directly call the API with the `KernelMode` value. Instead, the parameter will be set to the value of the calling thread's `PreviousMode` parameter, which is stored in the thread's kernel object structure. When you call a system call from a user-mode application, the `PreviousMode` value is set to `UserMode` and passed to any API that needs the `AccessMode` set.

Therefore, the kernel normally enforces all access checks. **Figure 7-1** shows the described behavior with a user-mode application calling the `NtCreateMutant` system call.

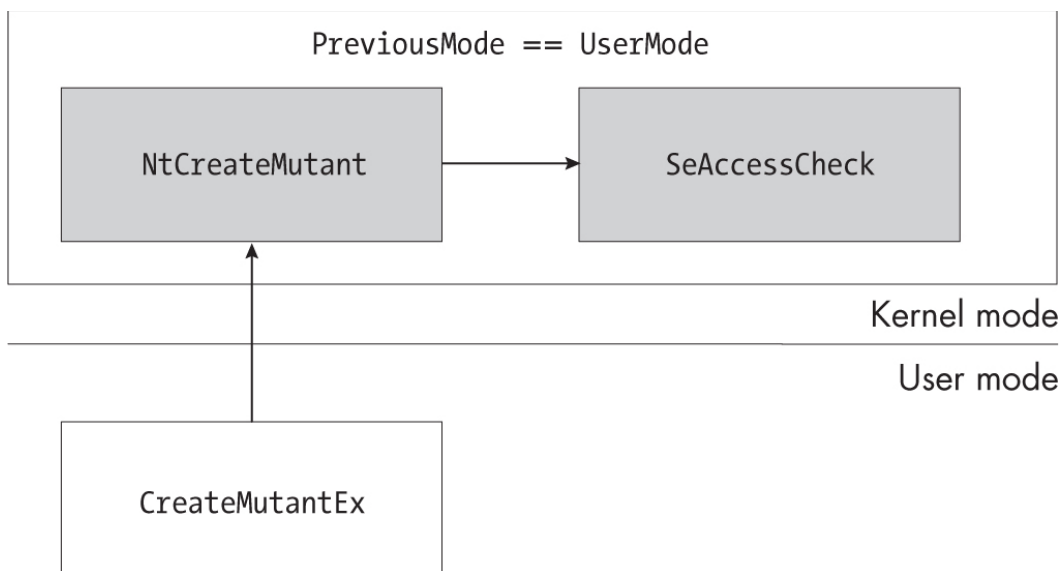


Figure 7-1: A thread's `PreviousMode` value when calling the `NtCreateMutant` system call

Even though the thread calling `SeAccessCheck` in [Figure 7-1](#) is executing kernel code, the thread's `PreviousMode` value reflects the fact that the call was started from `UserMode`. Therefore, the `AccessMode` parameter specified to `SeAccessCheck` will be `UserMode`, and the kernel will enforce the access check.

The most common way of transitioning the thread's `PreviousMode` value from `UserMode` to `KernelMode` is for the existing kernel code to call a system call via its `Zw` form: for example, `ZwCreateMutant`. When such a call is made, the system call dispatch correctly identifies that the previous execution occurred in the kernel and sets `PreviousMode` to `KernelMode`. [Figure 7-2](#) shows the transition of the thread's `PreviousMode` from `UserMode` to `KernelMode`.

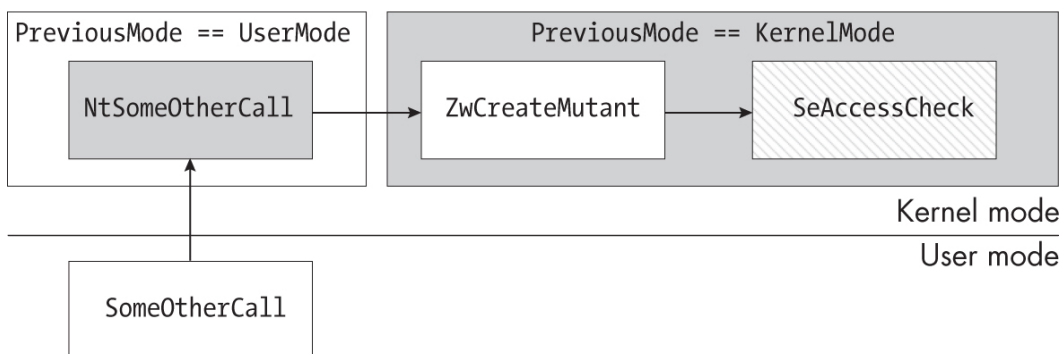


Figure 7-2: A thread's `PreviousMode` value being set to `KernelMode` after a call to `ZwCreateMutant`

In [Figure 7-2](#), the user-mode application calls a hypothetical kernel system call, `NtSomeOtherCall`, that internally calls `ZwCreateMutant`. The code executing in the `NtSomeOtherCall` function runs with the `PreviousMode` value set to

UserMode. However, once it calls `ZwCreateMutant`, the mode changes to `KernelMode` for the duration of the system call. In this case, because `ZwCreateMutant` would call `SeAccessCheck` to determine whether the caller had access to a `Mutant` object, the API would receive the `AccessMode` set to `KernelMode`, disabling access checking.

This behavior could introduce a security issue if the hypothetical `NtSomeOtherCall` allowed the user-mode application to influence where the `Mutant` object was created. Once the access check is disabled, it might be possible to create or modify the `Mutant` in a location that the user would not normally be allowed to access.

## Memory Pointer Checking

The access-mode parameter has a second purpose: when `UserMode` is specified, the kernel will check any pointers passed as parameters to a kernel API to ensure that they do not point to kernel memory locations. This is an important security restriction; it prevents an application in user mode from forcing a kernel API to read or write to kernel memory it should not have access to.

Specifying `KernelMode` disables these pointer checks at the same time as it disables the access checking. This mixing of behavior can introduce security issues: a kernel-mode driver might want to disable only pointer checking but inadvertently disable access checking as well.

How a caller can indicate these different uses of the access-mode parameter depends on the kernel APIs being used. For example, you can sometimes specify two `AccessMode` values, one for the pointer checking and one for the access checking. A more common method is to specify a flag to the call; for example, the `OBJECT_ATTRIBUTES` structure passed to system calls has a flag called `ForceAccessCheck` that disables pointer checking but leaves access checking enabled.

If you're analyzing a kernel driver, it's worth paying attention to the use of `Zw` APIs in which the `ForceAccessCheck` flag is not set. If a non-administrator user can control the target object manager path for the call, then there's likely to be a security vulnerability. For example, CVE-2020-17136

is a vulnerability in a kernel driver responsible for implementing the Microsoft OneDrive remote filesystem. The issue occurred because the API that the driver exposed to the Explorer shell did not set the `ForceAccessCheck` flag when creating a cloud-based file. Because of that, a user calling the APIs in the kernel driver could create an arbitrary file anywhere they wanted on the filesystem, allowing them to gain administrator privileges.

### ***User-Mode Access Checks***

To support user-mode applications, the kernel exposes its access check implementation through the `NtAccessCheck` system call. This system call uses the same access check algorithm as the `SeAccessCheck` API; however, it's tailored to the unique behavior of user-mode callers. The parameters for the system call are as follows:

**Security descriptor** The security descriptor to use for the check; must contain owner and group SIDs

**Client token** A handle to an impersonation token for the caller

**Desired access** An access mask for the access requested by the caller

**Generic mapping** The type-specific generic mapping

The API returns four values:

**Granted access** An access mask for the access the user was granted

**Access status code** An NT status code indicating the result of the access check

**Privileges** Any privileges used during the access check

**NT success code** A separate NT status code indicating the status of the system call

You'll notice that some of the parameters present in the kernel API are missing here. For example, there is no reason to specify the access mode,

as it will always be set to the caller's mode (UserMode, for a user-mode caller). Also, the caller's identity is now a handle to an impersonation token rather than a subject context. This handle must have `Query` access to be used for the access check. If you want to perform the access check against a primary token, you'll need to duplicate that token to an impersonation token first.

Another difference is that the impersonation token used in user mode can be as low as Identification level. The reason for this disparity is that the system call is designed for user services that want to check a caller's permissions, and it's possible that the caller will have granted access to an Identification-level token; this condition must be accounted for.

The system call also returns an additional NT status code instead of the Boolean value returned by the kernel API. The return value indicates whether there was a problem with the parameters passed to the system call. For example, if the security descriptor doesn't have both the owner and group SIDs set, the system call will return

`STATUS_INVALID_SECURITY_DESCR`.

### ***The Get-NtGrantedAccess PowerShell Command***

We can use the `NtAccessCheck` system call to determine the caller's granted access based on a security descriptor and an access token. The PowerShell module wraps the call to `NtAccessCheck` with the `Get-NtGrantedAccess` command, as shown in [Listing 7-1](#).

```
❶ PS> $sd = New-NtSecurityDescriptor -EffectiveToken -Type Mutant
PS> Format-NtSecurityDescriptor $sd -Summary
<Owner> : GRAPHITE\user
<Group> : GRAPHITE\None
<DACL>
GRAPHITE\user: (Allowed)(None)(Full Access)
NT AUTHORITY\SYSTEM: (Allowed)(None)(Full Access)
NT AUTHORITY\LogonSessionId_0_795805: (Allowed)(None)(ModifyState|...)

❷ PS> Get-NtGrantedAccess $sd -AsString
Full Access
```

```
❸ PS> Get-NtGrantedAccess $sd -Access ModifyState -AsString
ModifyState

❹ PS> Clear-NtSecurityDescriptorDacl $sd
PS> Format-NtSecurityDescriptor $sd -Summary
<Owner> : GRAPHITE\user
<Group> : GRAPHITE\None
<DACL> - <EMPTY>

PS> Get-NtGrantedAccess $sd -AsString
❺ ReadControl|WriteDac
```

### *Listing 7-1: Determining the caller's granted access*

We start by creating the default security descriptor using the `EffectiveToken` parameter ❶, and we confirm that it is correct by formatting it. In simplistic terms, the system call will check this security descriptor's DACL for an `Allowed` ACE that matches one of the token's SIDs; if such an ACE exists, it will grant the access mask. As the first ACE in the DACL grants the current user `SID Full Access`, we'd expect the result of the check to also grant `Full Access`.

We then call `Get-NtGrantedAccess`, passing it the security descriptor ❷. We don't specify an explicit token, so it uses the current effective token. We also do not specify an access mask, which means that the command checks `MaximumAllowed` access, converting the result to a string. It returns `Full Access`, as we expected based on the DACL.

Next, we test the `Get-NtGrantedAccess` command when supplied an explicit access mask using the `Access` parameter ❸. The command will work out the access mask enumeration for the security descriptor's type to allow us to specify type-specific values. We requested to check for `ModifyState`, so we receive only that access. For example, if we were opening a handle to a `Mutant` object, then the handle's access mask would grant only `ModifyState`.

Finally, to test an access denied case, we remove all the ACEs from the DACL ❹. If there is no `Allowed` ACE, then no access should be granted. But when we run `Get-NtGrantedAccess` again, we get a surprise: we were granted



ReadControl and WriteDac access instead of nothing ⑤. To understand why we received these access levels, we need to dig into the internals of the access check process. We'll do so in the next section.

## The Access Check Process in PowerShell

The access check process in Windows has changed substantially since the first version of Windows NT. This evolution has resulted in a complex set of algorithms that calculate what access a user is granted based on the combination of the security descriptor and the token. The flowchart in **Figure 7-3** shows the major components of the access check process.

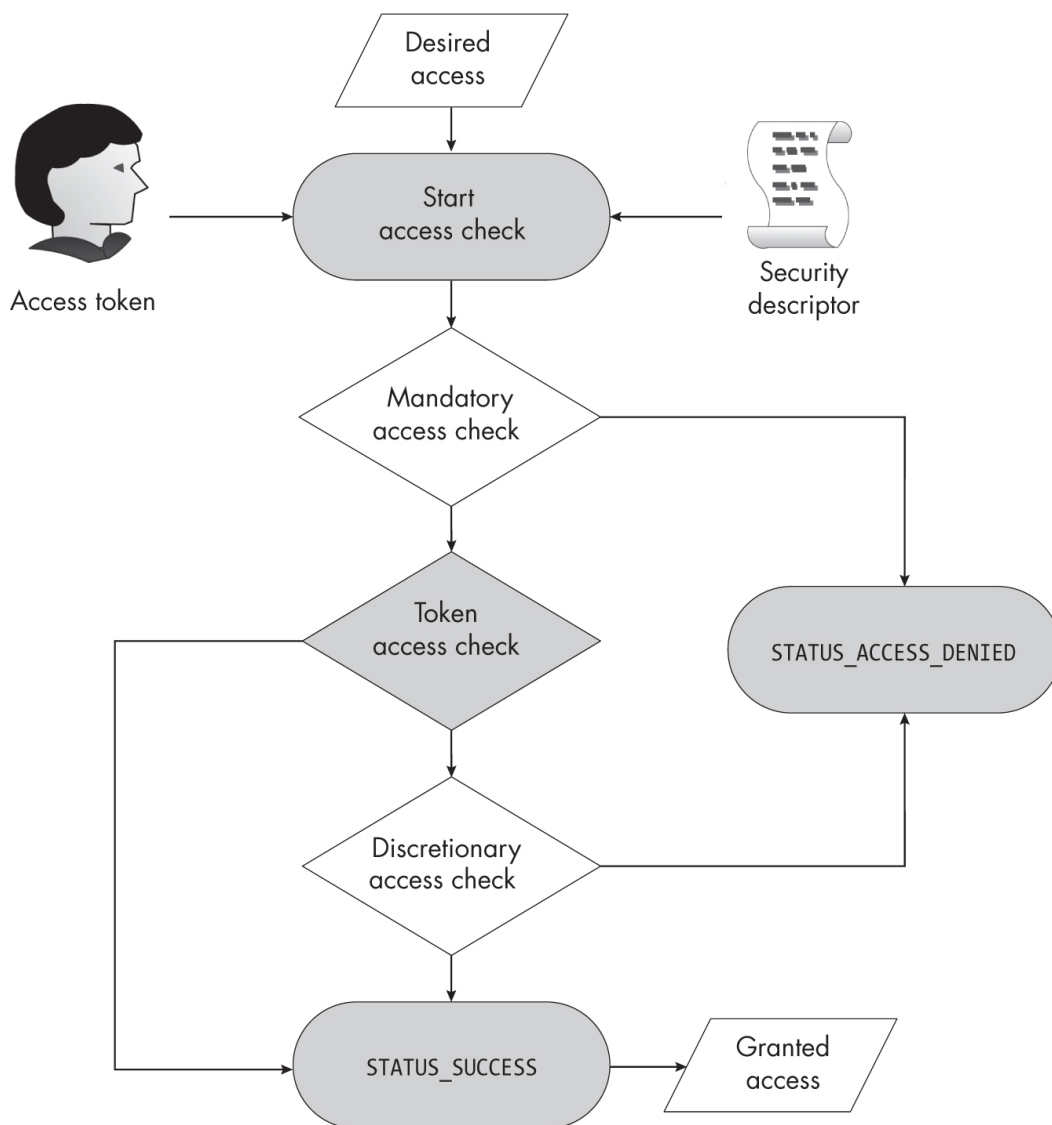


Figure 7-3: The access check process

The first step is to combine the token, the security descriptor, and the desired access mask. The access check process then uses this information in the following three main checks to determine whether access should be granted or denied:

**Mandatory access check** Denies access to resources when the token does not meet a set policy

**Token access check** Grants access based on the token's owner and privileges

**Discretionary access check** Grants or denies access based on the DACL

To explore these steps in more detail, let's write a basic implementation of the access check process in PowerShell. This PowerShell implementation won't replace the `Get-NtGrantedAccess` command, as, for simplicity, it won't check for maximum allowed access and might not include newer features. Even so, having an implementation that you can analyze and debug can help you gain a greater understanding of the overall process.

The implementation of the access check is quite complex; therefore, we'll build it in stages. You can access the full implementation in the *chapter7\_access\_check\_impl.psm1* script included with the book's example code. To use the script, import it as a module with this command:

```
PS> Import-Module .\chapter7_access_check_impl.psm1
```

### ***Defining the Access Check Function***

The module exports a single top-level function to perform the access check, `Get-PSGrantedAccess`, shown in [Listing 7-2](#).

```
function Get-PSGrantedAccess {  
    param(  
        $Token = (Get-NtToken -Effective -Pseudo),  
        $SecurityDescriptor,  
        $GenericMapping,  
        $DesiredAccess
```

```

    )

    ❶ $context = @{
        Token = $Token
        SecurityDescriptor = $SecurityDescriptor
        GenericMapping = $GenericMapping
        RemainingAccess = Get-NtAccessMask $DesiredAccess
        Privileges = @()
    }

    ## Test-MandatoryAccess defined below.
    ❷ if (!(Test-MandatoryAccess $context)) {
        return Get-AccessResult STATUS_ACCESS_DENIED
    }

    ## Get-TokenAccess defined below.
    Resolve-TokenAccess $context
    ❸ if (Test-NtAccessMask $context.RemainingAccess -Empty) {
        ❹ return Get-AccessResult STATUS_SUCCESS $context.Privileges
$DesiredAccess
    }

    ❺ if (Test-NtAccessMask $context.RemainingAccess AccessSystemSecurity) {
        return Get-AccessResult STATUS_PRIVILEGE_NOT_HELD
    }

    Get-DiscretionaryAccess $context
    ❻ if (Test-NtAccessMask $context.RemainingAccess -Empty) {
        return Get-AccessResult STATUS_SUCCESS $context.Privileges
$DesiredAccess
    }

    ❼ return Get-AccessResult STATUS_ACCESS_DENIED
}

```

*Listing 7-2: The top-level access check function*

The function accepts the four parameters we defined earlier in the chapter: a token, the security descriptor, the type's generic mapping, and the desired access. If the caller doesn't specify a token, we'll use their effective token for the rest of the access check.

The first task the function tackles is building a context that represents the current state of the access check process ❶. The most important property used here is `RemainingAccess`. We initially set this property to the `DesiredAccess` parameter, then remove bits from the property as they're granted during the access check process.

The rest of the function follows the flowchart in [Figure 7-3](#). First it performs the mandatory access check ❷. We'll describe what this check does in the next section. If the check fails, then the function completes with `STATUS_ACCESS_DENIED`. To simplify the code, the full script defines a helper function, `Get-AccessResult`, to build the result of the access check. [Listing 7-3](#) shows this function definition.

```
function Get-AccessResult {  
    param(  
        $Status,  
        $Privileges = @(),  
        $GrantedAccess = 0  
    )  
  
    $props = @{  
        Status = Get-NtStatus -Name $Status -PassStatus  
        GrantedAccess = $GrantedAccess  
        Privileges = $Privileges  
    }  
    return [PSCustomObject]$props  
}
```

*Listing 7-3: Implementing the `Get-AccessResult` helper function*

Next, the token access check updates the `RemainingAccess` property in the context ❸. If `RemainingAccess` becomes empty, then we can conclude we've been granted all access rights and can return `STATUS_SUCCESS` ❹. If it's not empty, we make a second check: if the caller requested `AccessSystemSecurity` and the token didn't grant that right, this check fails ❺.

Finally, we perform the discretionary access check. As with the token access check, we check the `RemainingAccess` property: if it's empty, the caller

has received all the accesses they've requested ⑥; otherwise, they've been denied access ⑦. With that overview in mind, let's delve into the details of each check in turn.

### ***Performing the Mandatory Access Check***

Windows Vista introduced a feature called *Mandatory Integrity Control* (MIC) that uses the token's integrity level and the mandatory label ACE to control resource access based on a general policy. MIC is a type of mandatory access check (MAC). The key behavior of a MAC is that it cannot grant access to a resource; it can only deny access. If the caller requests more access than the policy permits, the access check will immediately deny access, and if the MAC denies access, the DACL will never be checked. Because there is no way for a non-privileged user to circumvent the check, it's considered mandatory.

In the latest versions of Windows, the access check process performs two additional mandatory checks along with MIC. These checks implement similar behavior, so we'll group them together. [Listing 7-4](#) defines the `Test-MandatoryAccess` function we called in [Listing 7-2](#).

```
function Test-MandatoryAccess {  
    param($Context)  
  
    ## Test-ProcessTrustLevel is defined below.  
    if (!(Test-ProcessTrustLevel $Context)) {  
        return $false  
    }  
  
    ## Test-AccessFilter is defined below.  
    if (!(Test-AccessFilter $Context)) {  
        return $false  
    }  
  
    ## Test-MandatoryIntegrityLevel is defined below.  
    if (!(Test-MandatoryIntegrityLevel $Context)) {  
        return $false  
    }  
}
```

```
    return $true  
}
```

#### Listing 7-4: Implementing the Test-MandatoryAccess function

This function performs three checks: `Test-ProcessTrustLevel`, `Test-AccessFilter`, and `Test-MandatoryIntegrityLevel`. If any of these checks fails, then the entire access check process fails, returning `STATUS_ACCESS_DENIED`. Let's detail each check in turn.

### The Process Trust Level Check

Windows Vista introduced *protected processes*, which are processes that even an administrator can't manipulate and compromise. The original purpose of protected processes was to protect media content. However, Microsoft has since expanded them to cover a range of uses, such as protecting antivirus services and virtual machines.

A token can be assigned a *process trust level SID*. This SID depends on the protection level of a protected process and is assigned when such a process is created. To restrict access to a resource, the access check process determines whether the token's SID is equally or more trusted than a trust level SID in the security descriptor.

When one SID is considered equally or more trusted than another, it's said to *dominate*. To check whether one process trust level SID dominates another, you can call the `RtlSidDominatesForTrust` API or the `Compare-NtSid` command with the `Dominates` parameter. [Listing 7-5](#) translates the algorithm for checking the process trust level, which is stored in a process trust label ACE, into PowerShell.

```
function Test-ProcessTrustLevel {  
    param($Context)  
  
    ❶ $trust_level = Get-NtTokenSid $Token -TrustLevel  
    if ($null -eq $trust_level) {  
        $trust_level = Get-NtSid -TrustType None -TrustLevel None  
    }  
}
```

```

❷ $access = Get-NtAccessMask 0xFFFFFFFF
    $sacl = Get-NtSecurityDescriptorSacl $Context.SecurityDescriptor
    foreach($ace in $sacl) {
        ❸ if (!$ace.IsProcessTrustLabelAce -or $ace.IsInheritOnly) {
            continue
        }

        ❹ if (!(Compare-NtSid $trust_level $ace.Sid -Dominates)) {
            $access = Get-NtAccessMask $ace
        }
        break
    }

    $access = Grant-NtAccessMask $access AccessSystemSecurity
    ❺ return Test-NtAccessMask $access $Context.RemainingAccess -All
}

```

*Listing 7-5: The process trust level check algorithm*

To check the process trust level, we need to query the SID for the current token ❶. If the token does not have a trust level SID, then we define the lowest possible SID. Next, we initialize an access mask to all bits set ❷.

We then enumerate the values in the SACL, checking any process trust label ACE other than `InheritOnly` ❸. When we find a relevant ACE, we compare its SID to the SID queried for the token ❹. If the ACE SID dominates, then the token has a lower protection level, and the access mask is set to the value from the ACE.

Finally, we compare the access mask to the remaining access the caller requested ❺. If all the bits in the access mask are present in the remaining access, then the function returns `True`, which indicates that the process trust level check succeeded. Note that the check always adds `AccessSystemSecurity`, regardless of the mask in the ACE.

Let's test the behavior of the process trust label ACE. Rather than create a new protected process, we'll use the process trust level SID of the anonymous user's token for the access check. To simplify testing, we'll define a helper function that we can reuse. This function in [Listing 7-6](#) will create

a default security descriptor that grants access to both the current user and the anonymous user. Whenever we need a security descriptor for a test, we can call this function and use the returned value.

```
PS> function New-BaseSD {  
    $owner = Get-NtSid -KnownSid LocalSystem  
    $sd = New-NtSecurityDescriptor -Owner $owner -Group $owner -Type Mutant  
    Add-NtSecurityDescriptorAce $sd -KnownSid Anonymous -Access GenericAll  
    $sid = Get-NtSid  
    Add-NtSecurityDescriptorAce $sd -Sid $sid -Access GenericAll  
    Set-NtSecurityDescriptorIntegrityLevel $sd Untrusted  
    Edit-NtSecurityDescriptor $sd -MapGeneric  
    return $sd  
}
```

*Listing 7-6: Defining a helper function for testing*

The `New-BaseSD` function creates a basic security descriptor with the owner and group set to the `SYSTEM` user. It then adds an `Allowed` ACE for the anonymous and current user SIDs, granting them full access. It also sets the mandatory label to the `Untrusted` integrity level (you'll learn why the integrity level is important in "The Mandatory Integrity Level Check" on page 235). Finally, it maps any generic access to `Mutant` type-specific access. Let's now test the process trust label, as shown in [Listing 7-7](#).

```
❶ PS> $sd = New-BaseSD  
PS> $trust_sid = Get-NtSid -TrustType ProtectedLight -TrustLevel Windows  
PS> Add-NtSecurityDescriptorAce $sd -Type ProcessTrustLabel  
    -Access ModifyState -Sid $trust_sid  
PS> Get-NtGrantedAccess $sd -AsString  
❷ ModifyState  
  
❸ PS> $token = Get-NtToken -Anonymous  
PS> $anon_trust_sid = Get-NtTokenSid -Token $token -TrustLevel  
PS> Compare-NtSid $anon_trust_sid $trust_sid -Dominates  
❹ True  
PS> Get-NtGrantedAccess $sd -Token $token -AsString  
❺ Full Access
```



*Listing 7-7: Testing the process trust label ACE*

First, we create our base security descriptor and add a process trust label, granting `ModifyState` access only to tokens whose process trust level does not dominate the process trust label ❶. When we run the access check, we see that the effective token, which doesn't have any process trust level, gets `ModifyState` access only ❷, indicating that the process trust label is being enforced.

Next, we get a handle to an anonymous user's token using `Get-NtToken`, query its process trust level SID, and compare it to the SID we added to the security descriptor ❸. The call to `Compare-NtSid` returns `True` ❹, which indicates the token's process trust level SID dominates the one in the security descriptor. To confirm this, we run the access check and find that the anonymous user's token is granted `Full Access` ❺, which means the process trust label did not limit its access.

You might wonder whether you could impersonate the anonymous token to bypass the process trust label. Remember that in user mode we're calling `NtAccessCheck`, which takes only a single `Token` handle, but that the kernel's `SeAccessCheck` takes both a primary token and an impersonation token. Before the kernel verifies the process trust label, it checks both tokens and chooses the one with the lower trust level. Therefore, if the impersonation token is trusted but your primary token is untrusted, the effective trust level will be untrusted.

Windows applies a secondary security check when assigning the process trust label ACE to a resource. While you need only `WriteDac` access to set the process trust label, you cannot change or remove the ACE if your effective trust level does not dominate the label's trust level. This prevents you from setting a new, arbitrary process trust label ACE. Microsoft uses this ability to check certain files related to Windows applications for modifications and verify that the files were created by a protected process.

## The Access Filter ACE

The second mandatory access check is the access filter ACE. It works in a similar manner to the process trust label ACE, except that instead of using a process trust level to determine whether to apply a restricting access mask, it uses a conditional expression that evaluates to either `True` or `False`. If the conditional evaluates to `False`, the ACE's access mask limits the maximum granted access for the access check; if it evaluates to `True`, the access filter is ignored.

You can have multiple access filter ACEs in the SACL. Every conditional expression that evaluates to `False` removes more of the access mask. Therefore, if you match one ACE but don't match a second ACE that restricts to `GenericRead`, you'll get a maximum access of `GenericRead`. We can express this logic in a PowerShell function, as shown in [Listing 7-8](#).

```
function Test-AccessFilter {  
    param($Context)  
  
    $access = Get-NtAccessMask 0xFFFFFFFF  
    $sacl = Get-NtSecurityDescriptorSacl $Context.SecurityDescriptor  
    foreach($ace in $sacl) {  
        if (!$ace.IsAccessFilterAce -or $ace.IsInheritOnly) {  
            continue  
        }  
        ❶ if (!(Test-NtAceCondition $ace -Token $token)) {  
            ❷ $access = $access -band $ace.Mask  
        }  
    }  
  
    $access = Grant-NtAccessMask $access AccessSystemSecurity  
    ❸ return Test-NtAccessMask $access $Context.RemainingAccess -All  
}
```

*Listing 7-8: The access filter check algorithm*

This algorithm resembles the one we implemented to check the process trust level. The only difference is that we check a conditional expression rather than the SID ❶. The function supports multiple access filter ACEs;

for each matching ACE, the access mask is bitwise ANDed with the final access mask, which starts with all access mask bits set ❷. As the masks are ANDed, each ACE can only remove access, not add it. Once we've checked all the ACEs, we check the remaining access to determine whether the check succeeded or failed ❸.

In [Listing 7-9](#), we check the behavior of the access filter algorithm to ensure it works as expected.

```
PS> $sd = New-BaseSD
❶ PS> Add-NtSecurityDescriptorAce $sd -Type AccessFilter -KnownSid World
    -Access ModifyState -Condition "Exists TSA://ProcUnique" -MapGeneric
PS> Format-NtSecurityDescriptor $sd -Summary -SecurityInformation AccessFilter
<Access Filters>
Everyone: (AccessFilter)(None)(ModifyState)(Exists TSA://ProcUnique)

❷ PS> Show-NtTokenEffective -SecurityAttributes
SECURITY ATTRIBUTES
-----
Name                Flags                ValueType Values
----                -
TSA://ProcUnique    NonInheritable, Unique UInt64    {187, 365588953}

PS> Get-NtGrantedAccess $sd -AsString
❸ Full Access

PS> Use-NtObject($token = Get-NtToken -Anonymous) {
    Get-NtGrantedAccess $sd -Token $token -AsString
}
❹ ModifyState
```

*Listing 7-9: Testing the access filter ACE*

We add the access filter ACE to the security descriptor with the conditional expression "Exists TSA://ProcUnique" ❶. The expression checks whether the TSA://ProcUnique security attribute is present in the token. For a normal user, this check should always return `True`; however, the attribute doesn't exist in the anonymous user's token. We set the mask to `ModifyState` and the SID to the *Everyone* group. Note that the SID isn't veri-

fied, so it can have any value, but using the *Everyone* group is conventional.

We can check the current effective token's security attributes using `Show-NtTokenEffective` ❷. Getting the maximum access for the effective token results in `Full Access` ❸, meaning the access filter check passes without restricting access. However, when we repeat this using the anonymous user's token, the access filter check fails and the access is restricted to `ModifyState Only` ❹.

To set an access filter, you need only `WriteDac` access. So, what's to prevent a user removing the filter? Obviously, the access filter shouldn't grant `WriteDac` access in the first place, but if it does, you can limit any changes to a protected process trust level. To do this, set the ACE SID to a process trust level SID, and set the `TrustProtected` ACE flag. Now a caller with a lower process trust level won't be able to remove or modify the access filter ACE.

### The Mandatory Integrity Level Check

Finally, we'll implement the mandatory integrity level check. In the SACL, a mandatory label ACE's SID represents the security descriptor's integrity level. Its mask, which expresses the mandatory policy, combines the `NoReadUp`, `NoWriteUp`, and `NoExecuteUp` policies to determine the maximum access the system can grant the caller based on the `GenericRead`, `GenericWrite`, and `GenericExecute` values from the generic mapping structure.

To determine whether to enforce the policy, the check compares the integrity level SIDs of the security descriptor and token. If the token's SID dominates the security descriptor's, then no policy is enforced and any access is permitted. However, if the token's SID doesn't dominate, then any access requested outside of the value for the policy causes the access check to fail with `STATUS_ACCESS_DENIED`.

Calculating whether one integrity level SID dominates another is much simpler than calculating the equivalent value for the process trust level SID. To do so, we extract the last RID from each SID and compare these as

numbers. If one integrity level SID's RID is greater than or equal to the other, it dominates.

However, calculating the access mask for the policy based on the generic mapping is much more involved, as it requires a consideration of shared access rights. We won't implement the code for calculating the access mask, as we can use an option on `Get-NtAccessMask` to calculate it for us.

In **Listing 7-10**, we implement the mandatory integrity level check.

```
function Test-MandatoryIntegrityLevel {
    param($Context)

    $token = $Context.Token
    $sd = $Context.SecurityDescriptor
    $mapping = $Context.GenericMapping

    ❶ $policy = Get-NtTokenMandatoryPolicy -Token $token
    if (($policy -band "NoWriteUp") -eq 0) {
        return $true
    }

    if ($sd.HasMandatoryLabelAce) {
        $ace = $sd.GetMandatoryLabel()
        $sd_il_sid = $ace.Sid
        ❷ $access = Get-NtAccessMask $ace.Mask -GenericMapping $mapping
    } else {
        ❸ $sd_il_sid = Get-NtSid -IntegrityLevel Medium
        $access = Get-NtAccessMask -MandatoryLabelPolicy NoWriteUp
        -GenericMapping $GenericMapping
    }

    ❹ if (Test-NtTokenPrivilege -Token $token SeRelabelPrivilege) {
        $access = Grant-NtAccessMask $access WriteOwner
    }

    ❺ $il_sid = Get-NtTokenSid -Token $token -Integrity
    if (Compare-NtSid $il_sid $sd_il_sid -Dominates) {
        return $true
    }
}
```

```
    return Test-NtAccessMask $access $Context.RemainingAccess -All
}
```

*Listing 7-10: The mandatory integrity level check algorithm*

We start by checking the token's mandatory policy ❶. In this case, we check whether the `NoWriteUp` flag is set. If the flag is not set, then we disable integrity level checking for this token and return `True`. This flag is rarely turned off, however, and it requires `SeTcbPrivilege` to disable, so in almost all cases the integrity level check will continue.

Next, we need to capture the security descriptor's integrity level and mandatory policy from the mandatory label ACE. If the ACE exists, we extract these values and map the policy to the maximum access mask using `Get-NtAccessMask` ❷. If the ACE doesn't exist, the algorithm uses a `Medium` integrity level and a `NoWriteUp` policy by default ❸.

If the token has the `SeRelabelPrivilege` privilege, we add the `WriteOwner` access back to the maximum access, even if the policy removed it ❹. This allows a caller with `SeRelabelPrivilege` enabled to change the security descriptor's mandatory integrity label ACE.

We then query the token's integrity level SID and compare it to the security descriptor's ❺. If the token's SID dominates, then the check passes and allows any access. Otherwise, the calculated policy access mask must grant the entirety of the remaining access mask requested. Note that we don't treat `AccessSystemSecurity` differently here, as we did in the process trust level and access filter checks. We remove it if the policy contains `NoWriteUp`, the default for all resource types.

Let's verify the behavior of the mandatory integrity level check in the real access check process ([Listing 7-11](#)).

```
PS> $sd = New-BaseSD
PS> Format-NtSecurityDescriptor $sd -SecurityInformation Label -Summary
<Mandatory Label>
❶ Mandatory Label\Untrusted Mandatory Level: (MandatoryLabel)(None)(NoWriteUp)
```

```

PS> Use-NtObject($token = Get-NtToken -Anonymous) {
    Format-NtToken $token -Integrity
    Get-NtGrantedAccess $sd -Token $token -AsString
}
INTEGRITY LEVEL
-----
❷ Untrusted
Full Access

❸ PS> Remove-NtSecurityDescriptorIntegrityLevel $sd
PS> Use-NtObject($token = Get-NtToken -Anonymous) {
    Get-NtGrantedAccess $sd -Token $token -AsString
}
❹ ModifyState|ReadControl|Synchronize

```

*Listing 7-11: Testing the mandatory label ACE*

We first create a security descriptor and check its mandatory integrity label. We can see that it's set to the `Untrusted` integrity level, which is the lowest level, and that its policy is `NoWriteUp` ❶. We then get the maximum access for the anonymous user's token, which we can see has an integrity level of `Untrusted` ❷. As this integrity level matches the security descriptor's integrity level, the token is allowed full access.

To test access mask restrictions, we remove the mandatory label ACE from the security descriptor so that the access check will default to the `Medium` integrity level ❸. Running the check again, we now get `ModifyState|ReadControl|Synchronize` ❹, which is the `Mutant` object's full access without the `GenericWrite` access mask.

This concludes the implementation of the mandatory access check. We've seen that this algorithm is really composed of three separate checks for the process trust level, the access filter, and the integrity level. Each check can only deny access; it never grants additional access.

### ***Performing the Token Access Check***

The second main check, the token access check, uses properties of the caller's token to determine whether to grant certain access rights. More

specifically, it checks for any special privileges, as well as for the owner of the security descriptor.

Unlike the mandatory access check, the token access check can grant access to a resource if it has removed all bits from the token's access mask.

**Listing 7-12** implements the top-level `Result-TokenAccess` function.

```
Function Result-TokenAccess {  
    param($Context)  
  
    Resolve-TokenPrivilegeAccess $Context  
    if (Test-NtAccessMask $Context.RemainingAccess -Empty) {  
        return  
    }  
    return Resolve-TokenOwnerAccess $Context  
}
```

*Listing 7-12: The token access check algorithm*

The check is simple. First we check the token's privileges using a function we'll define next, `Resolve-TokenPrivilegeAccess`, passing it the current context. If certain privileges are enabled, this function modifies the token's remaining access; if the remaining access is empty, meaning no access remains to be granted, we can return immediately. We then call `Resolve-TokenOwnerAccess`, which checks whether the token owns the resource and can also update `RemainingAccess`. Let's dig into these individual checks.

### The Privilege Check

The *privilege check* (**Listing 7-13**) determines whether the `Token` object has three different privileges enabled. For each one, if the privilege is enabled we grant an access mask and the bits from the remaining access.

```
function Resolve-TokenPrivilegeAccess {  
    param($Context)  
  
    $token = $Context.Token  
    $access = $Context.RemainingAccess
```



```
❶ if ((Test-NtAccessMask $access AccessSystemSecurity) -and
    (Test-NtTokenPrivilege -Token $token SeSecurityPrivilege)) {
    $access = Revoke-NtAccessMask $access AccessSystemSecurity
    $Context.Privileges += "SeSecurityPrivilege"
}

❷ if ((Test-NtAccessMask $access WriteOwner) -and
    (Test-NtTokenPrivilege -Token $token SeTakeOwnershipPrivilege)) {
    $access = Revoke-NtAccessMask $access WriteOwner
    $Context.Privileges += "SeTakeOwnershipPrivilege"
}

❸ if ((Test-NtAccessMask $access WriteOwner) -and
    (Test-NtTokenPrivilege -Token $token SeRelabelPrivilege)) {
    $access = Revoke-NtAccessMask $access WriteOwner
    $Context.Privileges += "SeRelabelPrivilege"
}

❹ $Context.RemainingAccess = $access
}
```

*Listing 7-13: The token privilege access check algorithm*

First, we check whether the caller has requested `AccessSystemSecurity`; if so, and if `SeSecurityPrivilege` is enabled, we remove `AccessSystemSecurity` from the remaining access ❶. We also update the list of privileges we've used so that we can return it to the caller.

Next, we perform similar checks for `SeTakeOwnershipPrivilege` ❷ and `SeRelabelPrivilege` ❸ and remove `WriteOwner` from the remaining access if they're enabled. Lastly, we update the `RemainingAccess` value with the final access mask ❹.

Granting `WriteOwner` access to both `SeTakeOwnershipPrivilege` and `SeRelabelPrivilege` makes sense from the kernel's perspective, as you need `WriteOwner` access to modify the owner SID and integrity level. However, this implementation also means that a token with only `SeRelabelPrivilege` can take ownership of the resource, which we might not always intend. Fortunately, even administrators don't get `SeRelabelPrivilege` by default, making this a minor issue.

Let's check this function against the real access check process. Run the script in [Listing 7-14](#) as an administrator.

```

PS> $owner = Get-NtSid -KnownSid Null
❶ PS> $sd = New-NtSecurityDescriptor -Type Mutant -Owner $owner
    -Group $owner -EmptyDacl
❷ PS> Enable-NtTokenPrivilege SeTakeOwnershipPrivilege
❸ PS> Get-NtGrantedAccess $sd -Access WriteOwner -PassResult
    Status                Granted Access Privileges
    -----
❹ STATUS_SUCCESS         WriteOwner          SeTakeOwnershipPrivilege

❺ PS> Disable-NtTokenPrivilege SeTakeOwnershipPrivilege
PS> Get-NtGrantedAccess $sd -Access WriteOwner -PassResult
    Status                Granted Access Privileges
    -----
❻ STATUS_ACCESS_DENIED  None                NONE

```

*Listing 7-14: Testing the token privilege check*

We start by creating a security descriptor that should grant no access to the current user ❶. We then enable `SeTakeOwnershipPrivilege` ❷. Next, we request an access check for `WriteOwner` access and specify the `PassResult` parameter, which outputs the full access check result ❸. The result shows that the access check succeeded, granting `WriteOwner` access, but also that the check used the `SeTakeOwnershipPrivilege` ❹. To verify that we weren't granted `WriteOwner` access for another reason, we disable the privilege ❺ and rerun the check. This time, it denies us access ❻.

## The Owner Check

The *owner check* exists to grant `ReadControl` and `WriteDac` access to the owner of the resource, even if the DACL doesn't grant that owner any other access. The purpose of this check is to prevent a user from locking themselves out of their own resources. If they accidentally change the DACL so that they no longer have access, they can still use `WriteDac` access to return the DACL to its previous state.

The check compares the owner SID in the security descriptor with all enabled token groups (not just the token owner), granting access if a match is found. We demonstrated this behavior at the start of this chapter, in [Listing 7-1](#). In [Listing 7-15](#), we implement the `Resolve-TokenOwnerAccess` function.

```
function Resolve-TokenOwnerAccess {  
    param($Context)  
  
    $token = $Context.Token  
    $sd = $Context.SecurityDescriptor  
    $sd_owner = Get-NtSecurityDescriptorOwner $sd  
    ❶ if (!(Test-NtTokenGroup -Token $token -Sid $sd_owner.Sid)) {  
        return  
    }  
  
    ❷ $sids = Select-NtSecurityDescriptorAce $sd  
    -KnownSid OwnerRights -First -AclType Dacl  
    if ($sids.Count -gt 0) {  
        return  
    }  
  
    $access = $Context.RemainingAccess  
    ❸ $Context.RemainingAccess = Revoke-NtAccessMask $access ReadControl,  
    WriteDac  
}
```

*Listing 7-15: The token owner access check algorithm*

We use `Test-NtTokenGroup` to check whether the security descriptor's owner SID is an enabled member of the token ❶. If the owner SID is not a member, we simply return. If it is a member, the code then needs to check whether there are any *OWNER RIGHTS* SIDs (S-1-3-4) in the DACL ❷. If there are, then we don't follow the default process; instead, we rely on the DACL check to grant access to the owner. Finally, if both checks pass, we can remove `ReadControl` and `WriteDac` from the remaining access ❸.

In [Listing 7-16](#), we verify this behavior in the real access check process.

```
❶ PS> $owner = Get-NtSid -KnownSid World
PS> $sd = New-NtSecurityDescriptor -Owner $owner -Group $owner
-Type Mutant -EmptyDacL
PS> Get-NtGrantedAccess $sd
❷ ReadControl, WriteDac

❸ PS> Add-NtSecurityDescriptorAce $sd -KnownSid OwnerRights -Access ModifyState
PS> Get-NtGrantedAccess $sd
❹ ModifyState
```

*Listing 7-16: Testing the token owner check*

We start by creating a security descriptor with the owner and group set to *Everyone* ❶. We also create a security descriptor with an empty DACL, which means the access check process will consider only the owner check when calculating the granted access. When we run the access check, we get *ReadControl* and *WriteDac* ❷.

We then add a single ACE with the *OWNER RIGHTS* SID ❸. This disables the default owner access and causes the access check to grant only the access specified in the ACE (in this case, *ModifyState*). When we run the access check again, we now find that the only granted access is *ModifyState* ❹ and that we no longer have *ReadControl* or *WriteDac* access.

This concludes the token access check. As we demonstrated, the algorithm can grant certain access rights to a caller before any significant processing of the security descriptor takes place. This is primarily to allow users to maintain access to their own resources and for administrators to take ownership of other users' files. Now let's continue to the final check.

### ***Performing the Discretionary Access Check***

We've relied on the behavior of the DACL for a few of our tests. Now we'll explore exactly how the DACL check works. Checking the DACL may seem simple, but the devil is in the details. [Listing 7-17](#) implements the algorithm.

```

function Get-DiscretionaryAccess {
    param($Context)

    $token = $Context.Token
    $sd = $Context.SecurityDescriptor
    $access = $Context.RemainingAccess
    $resource_attrs = $null
    if ($sd.ResourceAttributes.Count -gt 0) {
        $resource_attrs = $sd.ResourceAttributes.ResourceAttribute
    }

    ❶ if (!(Test-NtSecurityDescriptor $sd -DaclPresent)
-or (Test-NtSecurityDescriptor $sd -DaclNull)) {
        $Context.RemainingAccess = Get-NtAccessMask 0
        return
    }

    $owner = Get-NtSecurityDescriptorOwner $sd
    $dacl = Get-NtSecurityDescriptorDacl $sd
    ❷ foreach($ace in $dacl) {
        ❸ if ($ace.IsInheritOnly) {
            continue
        }
        ❹ $sid = Get-AceSid $ace -Owner $owner
        $continue_check = $true
        switch($ace.Type) {
            "Allowed" {
                ❺ if (Test-NtTokenGroup -Token $token $sid) {
                    $access = Revoke-NtAccessMask $access $ace.Mask
                }
            }
            "Denied" {
                ❻ if (Test-NtTokenGroup -Token $token $sid -DenyOnly) {
                    if (Test-NtAccessMask $access $ace.Mask) {
                        $continue_check = $false
                    }
                }
            }
            "AllowedCompound" {
                $server_sid = Get-AceSid $ace -Owner $owner
                ❼ if ((Test-NtTokenGroup -Token $token $sid)
-and (Test-NtTokenGroup -Sid $server_sid)) {
                    $access = Revoke-NtAccessMask $access $ace.Mask
                }
            }
        }
    }
}

```

```

    }
}
"AllowedCallback" {
    ❸ if ((Test-NtTokenGroup -Token $token $sid)
-and (Test-NtAceCondition $ace -Token $token
-ResourceAttributes $resource_attrs)) {
        $access = Revoke-NtAccessMask $access $ace.Mask
    }
}

    ❹ if (!$continue_check -or (Test-NtAccessMask $access -Empty)) {
        break
    }
}

    ❺ $Context.RemainingAccess = $access
}

```

*Listing 7-17: The discretionary access check algorithm*

We begin by checking whether the DACL is present; if it is, we check whether it's a NULL ACL ❶. If there is no DACL or only a NULL ACL, there is no security to enforce, so the function clears the remaining access and returns, granting the token any access to the resource that the mandatory access check hasn't restricted.

Once we've confirmed that there is a DACL to check, we can enumerate each of its ACEs ❷. If an ACE is `InheritOnly`, it won't take part in the check, so we ignore it ❸. Next, we need to map the SID in the ACE to the SID we're checking using a helper function we'll define next, `Get-AceSid` ❹. This function converts the *OWNER RIGHTS* SID for the ACE to the current security descriptor's owner, as shown in [Listing 7-18](#).

```

function Get-AceSid {
    param(
        $Ace,
        $Owner
    )
}

```

```
$sid = $Ace.Sid
if (Compare-NtSid $sid -KnownSid OwnerRights) {
    $sid = $Owner.Sid
}

return $sid
}
```

*Listing 7-18: The implementation of Get-AceSid*

With the SID in hand, we can now evaluate each ACE based on its type. For the simplest type, `Allowed`, we check whether the SID is in the token's `Enabled` groups. If so, we grant the access represented by the ACE's mask and can remove those bits from the remaining access ⑤.

For the `Denied` type, we also check whether the SID is in the token's groups; however, this check must include both `Enabled` and `DenyOnly` groups, so we pass the `DenyOnly` parameter ⑥. Note that it's possible to configure the token's user SID as a `DenyOnly` group as well, and `Test-NtTokenGroup` takes this into account. A `Denied` ACE doesn't modify the remaining access; instead, the function compares the mask against the current remaining access, and if any bit of remaining access is also set in the mask, then the function denies that access and immediately returns the remaining access.

The final two ACE types we cover are variations on the `Allowed` type. The first, `AllowedCompound`, contains the additional server SID. To perform this check, the function compares both the normal SID and the server SID with the caller token's groups, as these values might be different ⑦. (Note that the server SID should be mapped to the owner if the *OWNER RIGHTS* SID is used.) The ACE condition is met only if both SIDs are enabled.

Finally, we check the `AllowedCallback` ACE type. To do so, we again check the SID, as well as whether a conditional expression matches the token using `Test-NtAceCondition` ⑧. If the expression returns `True`, the ACE condition is met, and we remove the mask from the remaining access. To fully implement the conditional check, we also need to pass in any resource attributes from the security descriptor (I'll describe resource attributes in more detail in "The Central Access Policy" on page 255). Notice that we're

intentionally not checking `DenyCallback`. This is because the kernel does not support `DenyCallback` ACEs, although the user mode-only `AuthzAccessCheck` API does.

After we've processed the ACE, we check the remaining access ⑨. If the remaining access is empty, we've been granted the entire requested access and can stop processing ACEs. This is why we have a canonical ACL ordering, as discussed in [Chapter 5](#); if `Denied` ACEs were placed after `Allowed` ACEs, the remaining access could become empty, and the loop might exit before ever checking a `Denied` ACE.

Lastly, this function sets the `RemainingAccess` ⑩. If the value of `RemainingAccess` is non-empty, the access check fails with `STATUS_ACCESS_DENIED`. Therefore, an empty DACL blocks all access; if there are no ACEs, the `RemainingAccess` never changes, so it won't be empty at the end of the function.

We've now covered all three access checks, and you should have a better understanding of their structure. However, there is more to the access check process. In the next section, we'll discuss how this process supports the implementation of sandboxes.

## Sandboxing

In [Chapter 4](#), we covered two types of sandbox tokens: restricted and lowbox. These sandbox tokens modify the access check process by adding more checks. Let's discuss each token type in more detail, starting with restricted tokens.

### *Restricted Tokens*

Using a restricted token affects the access check process by introducing a second owner and a discretionary access check against the list of restricted SIDs. In [Listing 7-19](#), we modify the owner SID check in the `Resolve-TokenOwnerAccess` function to account for this.

```
❶ if (!(Test-NtTokenGroup -Token $token -Sid $sd_owner.Sid)) {  
    return
```



```

    }

    if ($token.Restricted -and
❷ !(Test-NtTokenGroup -Token $token -Sid $sd_owner.Sid -Restricted)) {
        return
    }

```

*Listing 7-19: The modified Get-TokenOwner access check for restricted tokens*

We first perform the existing SID check ❶. If the owner SID isn't in the list of token groups, then we don't grant `ReadControl` or `WriteDac` access. Next is the additional check ❷: if the token is restricted, then we check the list of restricted SIDs for the owner SID and grant the token `ReadControl` and `WriteDac` access only if the owner SID is in both the main group list and the restricted SID list.

We'll follow the same pattern for the discretionary access check, although for simplicity, we'll add a Boolean `Restricted` switch parameter to the `Get-DiscretionaryAccess` function and pass it to any call to `Test-NtTokenGroup`. For example, we can modify the allowed ACE check implemented in [Listing 7-17](#), so it looks as shown in [Listing 7-20](#).

```

"Allowed" {
    if (Test-NtTokenGroup -Token $token $sid -Restricted:$Restricted) {
        $access = Revoke-NtAccessMask $access $ace.Mask
    }
}

```

*Listing 7-20: The modified Allowed ACE type for restricted tokens*

In [Listing 7-20](#), we set the `Restricted` parameter to the value of a parameter passed into `Get-DiscretionaryAccess`. We now need to modify the `Get-PSGrantedAccess` function defined in [Listing 7-2](#) to call `Get-DiscretionaryAccess` twice for a restricted token ([Listing 7-21](#)).

```

❶ $RemainingAccess = $Context.RemainingAccess
   Get-DiscretionaryAccess $Context
❷ $success = Test-NtAccessMask $Context.RemainingAccess -Empty

```

```
❸ if ($success -and $Token.Restricted) {  
    ❹ if (!$Token.WriteRestricted -or  
        (Test-NtAccessMask $RemainingAccess -WriteRestricted $GenericMapping)) {  
        $Context.RemainingAccess = $RemainingAccess  
        ❺ Get-DiscretionaryAccess $Context -Restricted  
        $success = Test-NtAccessMask $Context.RemainingAccess -Empty  
    }  
}  
  
❻ if ($success) {  
    return Get-AccessResult STATUS_SUCCESS $Context.Privileges $DesiredAccess  
}  
return Get-AccessResult STATUS_ACCESS_DENIED
```

*Listing 7-21: The `Get-PSGrantedAccess` function modified to account for restricted tokens*

We first capture the existing `RemainingAccess` value ❶, as the discretionary access check will modify it and we want to repeat that check a second time. We then run the discretionary access check and save the result in a variable ❷. If this first check succeeded and the token is restricted, we must perform a second check ❸. We also need to consider whether the token is write restricted and whether the remaining access includes write access ❹. We look for write access by checking the passed generic mapping. (Note that the owner check doesn't perform a write check, so in theory it could grant the token `WriteDac` access, which is considered a form of write access.)

Next we run the check again, this time with the `Restricted` parameter to indicate that the restricted SIDs should be checked ❺. If this second check also passes, we set the `$success` variable to `True` and grant access to the resource ❻.

Keep in mind that the restricted SID check applies to both `Allowed` and `Denied` ACE types. This means that if the DACL contains a `Denied` ACE that references a SID in the restricted SID list, the function will deny access, even if the SID isn't in the normal group list.

## Lowbox Tokens

The access check process for a lowbox token resembles that for a restricted token. A lowbox token can contain a list of capability SIDs used to perform a second check, like the check we performed with the list of restricted SIDs. Likewise, if the access check process doesn't grant access through both normal and capability checks, the access check fails. However, the lowbox token's access check contains some subtle differences:

- It will consider the token's package SID in addition to its list of capability SIDs.
- The checked capability SIDs must have the enabled attribute flag set to be considered active.
- The check applies only to `Allowed` ACE types, not to `Denied` ACE types.
- `NULL` DACLs do not grant full access.

In addition, two special package SIDs will match any token's package SID for the purposes of the package SID check:

- *ALL APPLICATION PACKAGES* (S-1-15-2-1)
- *ALL RESTRICTED APPLICATION PACKAGES* (S-1-15-2-2)

Checking for the *ALL APPLICATION PACKAGES* SID during the package SID check can be disabled if the token used for the access check has the `WIN: //NOALLAPPPKG` security attribute set to a single value of 1. In this case, the package SID check will only consider the *ALL RESTRICTED APPLICATION PACKAGES* SID. If the security attribute isn't present or is set to 0, the access check considers both special package SIDs. Microsoft refers to processes with this security attribute as running a *Less Privileged AppContainer (LPAC)*.

Because setting a token's security attribute requires the `SeTcbPrivilege` privilege, the process creation APIs have an option for adding the `WIN: //NOALLAPPPKG` security attribute to a new process's token. [Listing 7-22](#) shows a basic implementation of the lowbox access check for `Allowed` ACE types. You should add this code to the discretionary access check in [Listing 7-17](#), in the locations indicated in the comments.

```

## Add to start of Get-DiscretionaryAccess.
$ac_access = $context.DesiredAccess
if (!$token.AppContainer) {
    $ac_access = Get-NtAccessMask 0
}

## Replace the Allowed case in the ACE switch statement.
"Allowed" {
    if (Test-NtTokenGroup -Token $token $sid -Restricted:$Restricted) {
        ❶ $access = Revoke-NtAccessMask $access $ace.Mask
    } else {
        ❷ if ($Restricted) {
            break
        }

        ❸ if (Test-NtTokenGroup -Token $token $sid -Capability) {
            ❹ $ac_access = Revoke-NtAccessMask $ac_access $ace.Mask
        }
    }
}

## Add at end of ACE loop.
❺ $effective_access = $access -bor $ac_access

```

*Listing 7-22: An implementation of the lowbox access check for Allowed ACEs*

The first test verifies whether the SID is in the token's group list. If it finds the SID in the group list, it removes the mask from the remaining access check ❶. If the group test fails, we check whether it's a package or capability SID. We must ensure that we're not checking whether we're in the restricted SID mode ❷, as this mode doesn't define lowbox checks.

Our check for the capability SIDs includes the package SID and the *ALL APPLICATION PACKAGES* SID ❸. If we find a match, we remove the mask from the remaining access ❹. However, we need to maintain separate remaining access values for normal SIDs and AppContainer SIDs. Therefore, we create two variables, `$access` and `$ac_access`. We initialize the `$ac_access` variable to the value of the original `DesiredAccess`, not the current remaining access, as we won't grant owner rights such as `WriteDac` unless the SID

also matches an `Allowed` package or capability SID ACE. We also modify the loop's exit condition to consider both remaining access values ❹; they must both be empty before we exit.

Next, we'll add some additional checks to better isolate AppContainer processes from existing `Low` integrity level sandboxes, such as Internet Explorer's protected mode. The first change we implement affects the mandatory access check. If the check fails for a lowbox token, we then check the security descriptor's integrity level a second time. If the integrity level is less than or equal to `Medium`, we assume that the check succeeds. This is even though lowbox tokens have a `Low` integrity level, as demonstrated in [Chapter 4](#), which would normally prevent write access to the resource. This behavior allows a more privileged application to grant a lowbox token access to a resource while blocking `Low` integrity level sandboxes.

**Listing 7-23** demonstrates this behavior.

```
❶ PS> $sd = New-NtSecurityDescriptor -Owner "BA" -Group "BA" -Type Mutant
PS> Add-NtSecurityDescriptorAce $sd -KnownSid World -Access GenericAll
PS> Add-NtSecurityDescriptorAce $sd -KnownSid AllApplicationPackages
    -Access GenericAll
PS> Edit-NtSecurityDescriptor $sd -MapGeneric
❷ PS> Set-NtSecurityDescriptorIntegrityLevel $sd Medium

PS> Use-NtObject($token = Get-NtToken -Duplicate -IntegrityLevel Low) {
    Get-NtGrantedAccess $sd -Token $token -AsString
}
❸ ModifyState|ReadControl|Synchronize

PS> $sid = Get-NtSid -PackageName "mandatory_access_lowbox_check"
PS> Use-NtObject($token = Get-NtToken -LowBox -PackageSid $sid) {
    Get-NtGrantedAccess $sd -Token $token -AsString
}
❹ Full Access
```

*Listing 7-23: The behavior of a mandatory access check against a lowbox token*

We start by building a security descriptor that grants `GenericAll` access for the *Everyone* and *ALL APPLICATION PACKAGES* groups ❶. We also set an explicit integrity level of `Medium` ❷, although this isn't necessary, as `Medium` is the default for security descriptors without a mandatory label ACE. We then perform an access check using a `Low` integrity level token, and we receive only read access to the security descriptor ❸. Next, we try the access check again with a lowbox token; although the token's integrity level is still `Low`, the token is granted `Full Access` ❹.

The second change we implement is that if the DACL contains a package SID we deny access to the `Low` integrity level token, regardless of the security descriptor's integrity level or DACL. This mechanism blocks access to resources that are assigned the default DACL, as the package SID is added to the default DACL when a lowbox token is created. [Listing 7-24](#) tests this behavior.

```
PS> $sid = Get-NtSid -PackageName 'package_sid_low_il_test'
❶ PS> $token = Get-NtToken -LowBox -PackageSid $sid
❷ PS> $sd = New-NtSecurityDescriptor -Token $token -Type Mutant
PS> Format-NtSecurityDescriptor $sd -Summary -SecurityInformation Dacl, Label
<DACL>
❸ GRAPHITE\user: (Allowed)(None)(Full Access)
NT AUTHORITY\SYSTEM: (Allowed)(None)(Full Access)
NT AUTHORITY\LogonSessionId_0_109260: (Allowed)(None)(ModifyState|...)
❹ package_sid_low_il_test: (Allowed)(None)(Full Access)
<Mandatory Label>
❺ Mandatory Label\Low Mandatory Level: (MandatoryLabel)(None)(NoWriteUp)

PS> Get-NtGrantedAccess $sd -Token $token -AsString
❻ Full Access

PS> $token.Close()
PS> $low_token = Get-NtToken -Duplicate -IntegrityLevel Low
PS> Get-NtGrantedAccess $sd -Token $low_token -AsString
❼ None
```

*Listing 7-24: Verifying the behavior of the package SID for Low integrity level tokens*

We start by creating a lowbox token ❶. The token does not have any added capability SIDs, only the package SID. Next, we build a default security descriptor from the lowbox token ❷. When inspecting the entries in the security descriptor, we see that the current user SID ❸ and the package SID ❹ have been granted `Full Access`. As a lowbox token has `Low` integrity level, the security descriptor inheritance rules require the integrity level to be added to the security descriptor ❺.

We then request the granted access for the security descriptor based on the lowbox token and receive `Full Access` ❻. Next, we create a duplicate of the current token but set its integrity level to `Low`. We now get a granted access of `None` ❼, even though we expected to receive `Full Access` based on the integrity level ACE in the security descriptor. In this case, the presence of the package SID in the security descriptor blocked access.

One final thing to note: as the sandbox access checks are orthogonal, it's possible to create a lowbox token from a restricted token, causing both lowbox checks and restricted SID checks to occur. The resulting access is the most restrictive of all, making for a stronger sandbox primitive.

## Enterprise Access Checks

Enterprise deployments of Windows often perform some additional access checks. You won't typically need these checks on stand-alone installations of Windows, but you should still understand how they modify the access check process if present.

### *The Object Type Access Check*

For simplicity's sake, one thing I intentionally removed from the discretionary access check algorithm was the handling of object ACEs. To support object ACEs, you must use a different access check API: either `SeAccessCheckByType` in kernel mode or the `NtAccessCheckByType` system call. These APIs introduce two additional parameters to the access check process:

**Principal** A SID used to replace the *SELF* SID in ACEs

**objectTypes** A list of GUIDs that are valid for the check

The `Principal` is easy to define: when we're processing the DACL and we encounter an ACE's SID that's set to the *SELF* SID (S-1-5-10), we replace the SID with a value from the `Principal` parameter. (Microsoft introduced the *SELF* SID for use in Active Directory; we'll discuss its purpose in more detail in [Chapter 11](#).) [Listing 7-25](#) shows an adjusted version of the `Get-AceSid` function that takes this into account. You'll also have to modify the `Get-PSGrantedAccess` function to receive the `Principal` parameter by adding it to the `$Context` value.

```
function Get-AceSid {
    Param (
        $Ace,
        $Owner,
        $Principal
    )

    $sid = $Ace.Sid
    if (Compare-NtSid $sid -KnownSid OwnerRights) {
        $sid = $Owner
    }
    if ((Compare-NtSid $sid -KnownSid Self) -and ($null -NE $Principal)) {
        $sid = $Principal
    }
    return $sid
}
```

*Listing 7-25: Adding the principal SID to the `Get-AceSid` function*

[Listing 7-26](#) tests the behavior of the `Principal` SID.

```
PS> $owner = Get-NtSid -KnownSid LocalSystem
❶ PS> $sd = New-NtSecurityDescriptor -Owner $owner -Group $owner -Type Mutant
PS> Add-NtSecurityDescriptorAce $sd -KnownSid Self -Access GenericAll
    -MapGeneric
❷ PS> Get-NtGrantedAccess $sd -AsString
None

PS> $principal = Get-NtSid
```



```
❸ PS> Get-NtGrantedAccess $sd -Principal $principal -AsString  
Full Access
```

*Listing 7-26: Testing the Principal SID replacement*

We start by creating a security descriptor with the owner and group set to the *SYSTEM* user SID and a single *Allowed* ACE that grants the *SELF* SID *GenericAll* access ❶. Based on the access-checking rules, this should not grant the user any access to the resource. We can confirm that this is the case with a call to *Get-NtGrantedAccess* ❷.

Next, we get the effective token's user SID and pass it in the *Principal* parameter to *Get-NtGrantedAccess* ❸. The DACL check will then replace the *SELF* SID with the *Principal* SID, which matches the current user and therefore grants *Full Access*. This check replaces SIDs in the DACL and SACL only; setting *SELF* as the owner SID won't grant any access.

The other parameter, *ObjectTypes*, is much trickier to implement. It provides a list of GUIDs that are valid for the access check process. Each GUID represents the type of an object to be accessed; for example, you might have a GUID associated with a computer object and a different one for a user object.

Each GUID also has an associated level, turning the list into a hierarchical tree. Each node maintains its own remaining access, which it initializes to the main *RemainingAccess* value. Active Directory uses this hierarchy to implement a concept of properties and property sets, as shown in [Figure 7-4](#).

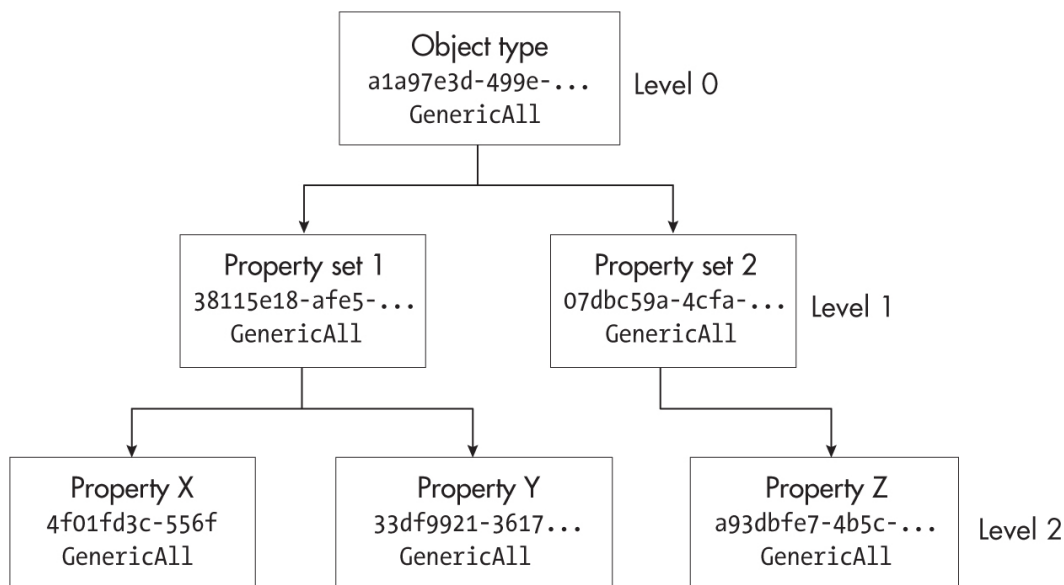


Figure 7-4: Active Directory-style properties

Each node in **Figure 7-4** shows the name we've given it, a portion of the `ObjectType` GUID, and the current `RemainingAccess` value (in this case, `GenericAll`). Level 0 corresponds to the top-level object, of which there can be only one in the list. At level 1 are the property sets, here numbered 1 and 2. Below each property set, at level 2, are the individual properties.

Setting up the object types in a hierarchy enables us to configure a security descriptor to grant access to multiple properties using a single ACE by setting the access on the property set. If we grant a property set some access, we also grant that access to all properties contained in that set. Conversely, if we deny access to a single property, the deny status will propagate up the tree and deny access to the entire property set and object as a whole.

Let's consider a basic implementation of object type access. The code in **Listing 7-27** relies on an `ObjectTypes` property added to the access context. We can generate the values for this parameter using the `New-ObjectTypeTree` and `Add-ObjectTypeTree` commands, whose use we'll cover on page 254.

**Listing 7-27** shows the access check implementation for the `AllowedObject` ACE type. Add it to the ACE enumeration code from **Listing 7-17**.

```

"AllowedObject" {
  ❶ if (!(Test-NtTokenGroup -Token $token $sid)) {

```

```
        break
    }

    ❷ if ($null -eq $Context.ObjectTypes -or $null -eq $ace.ObjectType) {
        break
    }

    ❸ $object_type = Select-ObjectTypeTree $Context.ObjectTypes
    if ($null -eq $object_type) {
        break
    }

    ❹ Revoke-ObjectTypeTreeAccess $object_type $ace.Mask
    $access = Revoke-NtAccessMask $access $ace.Mask
}
```

*Listing 7-27: An implementation of the AllowedObject ACE access check algorithm*

We start with the SID check ❶. If the SIDs don't match, we don't process the ACE. Next, we check whether the `ObjectTypes` property exists in the context and whether the ACE defines an `ObjectType` ❷ (the `ObjectType` on the ACE is optional). Again, if these checks fail, we ignore the ACE. Finally, we check whether there is an entry in the `ObjectTypes` parameter for the `ObjectType` GUID ❸.

If all checks pass, we consider the ACE for the access check. First we revoke the access from the entry in the tree of objects ❹. This removes the access not only from the `ObjectType` entry we found but also from any children of that entry. We also revoke the access we're maintaining for this function.

Let's apply this behavior to the tree shown in [Figure 7-4](#). If the `AllowedObject` ACE grants `GenericAll` access to property set 1, the new tree will look like the one in [Figure 7-5](#).

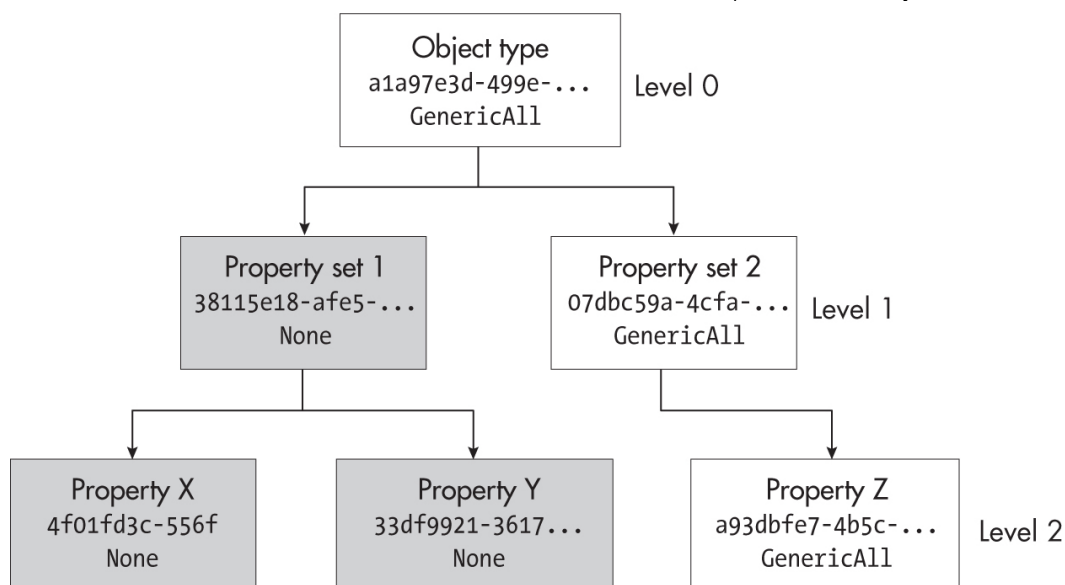


Figure 7-5: The object type tree after access is granted to property set 1

As the `GenericAll` access has been removed from the `RemainingAccess` for property set 1, it's also been removed for properties X and Y. These nodes now have an empty `RemainingAccess`. Note that for `Allowed` ACEs only the main `RemainingAccess` matters, as the tree's purpose is to handle `Denied` ACEs correctly. This means that not every object type must have a `RemainingAccess` of `0` for the access check to succeed.

Now let's handle the `DeniedObject` ACE. Add the code in [Listing 7-28](#) to the existing ACE enumeration code in [Listing 7-17](#).

```

"DeniedObject" {
  ❶ if (!(Test-NtTokenGroup -Token $token $sid -DenyOnly)) {
    break
  }

  ❷ if ($null -ne $Context.ObjectTypes) {
    if ($null -eq $ace.ObjectType) {
      break;
    }

    $object_type = Select-ObjectTypeTree $Context.ObjectTypes
    $ace.ObjectType
    if ($null -eq $object_type) {
      break
    }
  }
}
  
```

```
    ❸ if (Test-NtAccessMask $object_type.RemainingAccess $ace.Mask) {  
        $continue_check = $false  
        break  
    }  
}  
❹ if (Test-NtAccessMask $access $ace.Mask) {  
    $continue_check = $false  
}  
}
```

*Listing 7-28: An implementation of the DeniedObject ACE access check algorithm*

As usual, we begin by checking all ACEs with the DeniedObject type ❶. If the check passes, we next check the ObjectTypes context property ❷. When we handled the AllowedObject ACE, we stopped the check if the ObjectType property was missing. However, we handle the DeniedObject ACEs differently. If there is no ObjectTypes property, the check will continue as if it were a normal Denied ACE, by considering the main RemainingAccess ❹.

If the ACE's access mask contains bits in the RemainingAccess, we deny access ❸. If this check passes, we check the value against the main RemainingAccess. This demonstrates the purpose of maintaining the tree: if the Denied ACE matched property X in [Figure 7-5](#), the denied mask would have no effect. However, if the Denied ACE matched property Z, then that object type, and by association property set 2 and the root object type, would be denied as well. [Figure 7-6](#) demonstrates this: you can see that those nodes are all now denied, even though the property set 1 branch is still allowed.

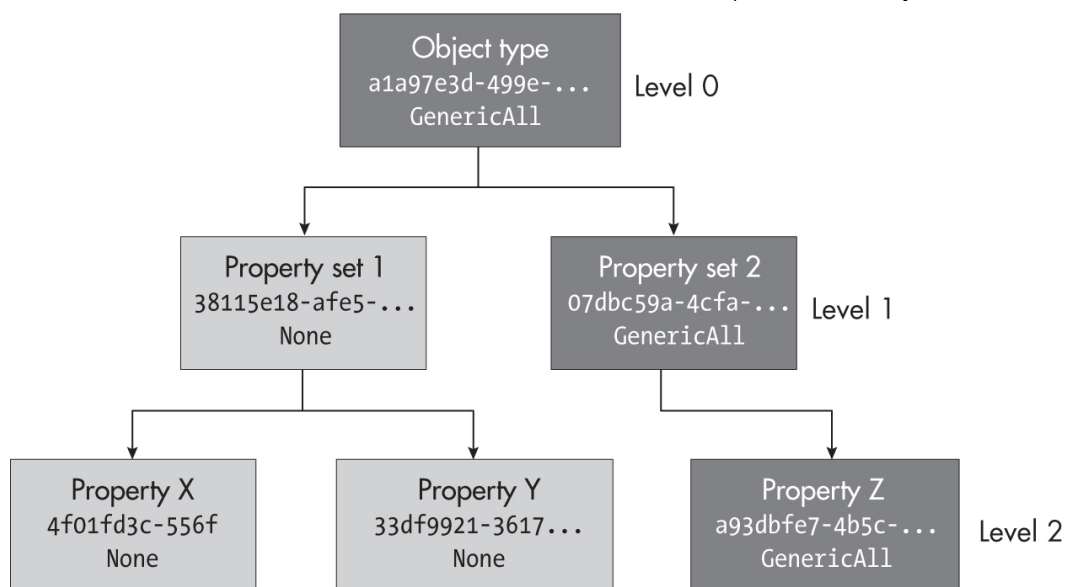


Figure 7-6: The object type tree after denying access to property Z

The `NtAccessCheckByType` system call returns a single status and granted access for the entire list of object types, reflecting the access specified at the root of the object type tree. Therefore, in the case of [Figure 7-6](#), the whole access check would fail.

To figure out which particular object types failed the access check, you can use the `NtAccessCheckByTypeResultList` system call, which returns a status and the granted access for every entry in the object type list. [Listing 7-29](#) shows how you can use this system call by specifying the `ResultList` parameter to `Get-NtGrantedAccess`.

```

❶ PS> $tree = New-ObjectTypeTree (New-Guid) -Name "Object"
PS> $set_1 = Add-ObjectTypeTree $tree (New-Guid) -Name "Property Set 1"
-PassThru
PS> $set_2 = Add-ObjectTypeTree $tree (New-Guid) -Name "Property Set 2"
-PassThru
PS> Add-ObjectTypeTree $set_1 (New-Guid) -Name "Property X"
PS> Add-ObjectTypeTree $set_1 (New-Guid) -Name "Property Y"
PS> $prop_z = New-Guid
PS> Add-ObjectTypeTree $set_2 $prop_z -Name "Property Z"

PS> $owner = Get-NtSid -KnownSid LocalSystem
PS> $sd = New-NtSecurityDescriptor -Owner $owner -Group $owner -Type Mutant
❷ PS> Add-NtSecurityDescriptorAce $sd -KnownSid World -Access WriteOwner
-MapGeneric -Type DeniedObject -ObjectType $prop_z
PS> Add-NtSecurityDescriptorAce $sd -KnownSid World
  
```

```

-Access ReadControl, WriteOwner -MapGeneric
PS> Edit-NtSecurityDescriptor $sd -CanonicalizeDacl
❸ PS> Get-NtGrantedAccess $sd -PassResult -ObjectType $tree
-Access ReadControl, WriteOwner | Format-Table Status, SpecificGrantedAccess,
Name

```

|                        | Status | SpecificGrantedAccess | Name   |
|------------------------|--------|-----------------------|--------|
|                        | -----  | -----                 | ----   |
| ❹ STATUS_ACCESS_DENIED |        | None                  | Object |

```

❺ PS> Get-NtGrantedAccess $sd -PassResult -ResultList -ObjectType $tree
-Access ReadControl, WriteOwner | Format-Table Status, SpecificGrantedAccess,
Name

```

|                      | ❻ Status | SpecificGrantedAccess   | Name           |
|----------------------|----------|-------------------------|----------------|
|                      | -----    | -----                   | ----           |
| STATUS_ACCESS_DENIED |          | ReadControl             | Object         |
| STATUS_SUCCESS       |          | ReadControl, WriteOwner | Property Set 1 |
| STATUS_SUCCESS       |          | ReadControl, WriteOwner | Property X     |
| STATUS_SUCCESS       |          | ReadControl, WriteOwner | Property Y     |
| STATUS_ACCESS_DENIED |          | ReadControl             | Property Set 2 |
| STATUS_ACCESS_DENIED |          | ReadControl             | Property Z     |

*Listing 7-29: Example showing the difference between normal and list results*

We start by building the object type tree to match the tree in [Figure 7-4](#) ❶. We don't care about the specific GUID values except for that of property Z, which we'll need for the DeniedObject ACE, so we generate random GUIDs. Next, we build the security descriptor, creating an ACE that denies ReadControl access to property Z ❷. We also include a non-object ACE to grant ReadControl and WriteOwner access.

We first run the access check with the object type tree but without the ResultList parameter, requesting both ReadControl and WriteOwner access ❸. We use the Denied ACE, as it matches an ObjectType GUID in the object type tree. As we expected, this causes the access check process to return STATUS\_ACCESS\_DENIED, with None as the granted access ❹.

When we execute the access check again, this time with ResultList, we receive a list of access check results ❺. The top-level object entry still indicates that access was denied, but access was granted to property set 1 and

its children ⑥. This result corresponds to the tree shown in [Figure 7-6](#). Also note that the entries for which access was denied don't show an empty granted access; instead, they indicate that `ReadControl` access would have been granted if the request had succeeded. This is an artifact of how the access check is implemented under the hood and almost certainly shouldn't be used.

### ***The Central Access Policy***

The *central access policy*, a feature added in Windows 8 and Windows Server 2012 for use in enterprise networks, is the core security mechanism behind a Windows feature called *Dynamic Access Control*. It relies on device and user claim attributes in the token.

We talked briefly about user and device claims in [Chapter 4](#), when discussing the conditional expression format. A *user claim* is a security attribute added to the token for a specific user. For example, you might have a claim that represents the country in which a user is employed. You can sync the value of the claim with values stored in Active Directory so that if the user, say, moves to another country, their user claim will update the next time they authenticate.

A *device claim* belongs to the computer used to access the resource. For example, a device claim might indicate whether the computer is located in a secure room or is running a specific version of Windows. [Figure 7-7](#) shows a common use of a central access policy: restricting access to files on a server in an enterprise network.



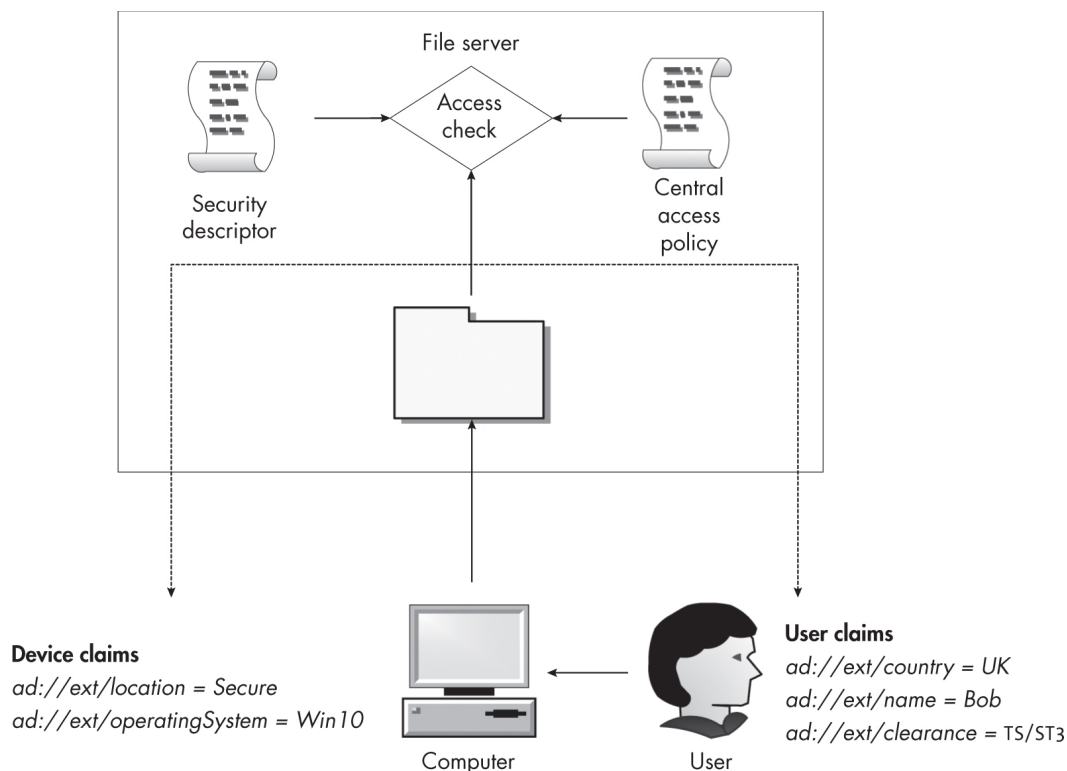


Figure 7-7: A central access policy on a file server

This central access policy contains one or more security descriptors that the access check will consider in addition to a file's security descriptor. The final granted access is the most restrictive result of the access checks. While not strictly necessary, the additional security descriptors can rely on user and device claims in `AllowedCallback` ACEs to determine the granted access. The enterprise's Kerberos authentication must be configured to support the claims in order to send them over the network. We'll come back to Kerberos authentication in [Chapter 14](#).

You might wonder how using a central access policy differs from simply configuring the security of the files to use the device and user claims. The main difference is that it's managed centrally using policies in the enterprise domain group policy. This means an administrator can change the central access policy in one place to update it across the enterprise.

A second difference is that the central access policy works more like a mandatory access control mechanism. For example, a user might typically be able to modify the security descriptor for the file; however, the central access policy could restrict their access or block it outright if, for

example, the user moved to a new country or used a different computer not accounted for in the rules.

We won't discuss how to configure a central access policy, as that topic is more appropriate for a book on Windows enterprise management.

Instead, we'll explore how it's enforced by the kernel's access check process. The Windows registry stores the central access policy when the computer's group policy is updated, and you can find the key at the following location:

*HKEY\_LOCAL\_MACHINE\SYSTEM\CurrentControlSet\Control\Lsa\CentralizedAccessPolicies.*

There can be more than one configured policy, each containing the following information:

- The name and description of the policy
- A SID that uniquely identifies the policy
- One or more policy rules

In turn, each policy rule contains the following information:

- The name and description of the rule
- A conditional expression that determines when the rule should be enforced
- The security descriptor to use in the central access policy access check
- An optional staging security descriptor used to test new policy rules

You can use the `Get-CentralAccessPolicy` PowerShell command to display the list of policies and rules. For most Windows systems, the command won't return any information. To see results like those in [Listing 7-30](#), you'll need to join a domain that is configured to use a central access policy.

```
PS> Get-CentralAccessPolicy
```

| Name               | CapId                            | Description               |
|--------------------|----------------------------------|---------------------------|
| -----              | -----                            | -----                     |
| Secure Room Policy | S-1-17-3260955821-1180564752-... | Only for Secure Computers |
| Main Policy        | S-1-17-76010919-1187351633-...   |                           |

```
PS> $rules = Get-CentralAccessPolicy | Select-Object -ExpandProperty Rules
```

```

PS> $rules | Format-Table
Name           Description AppliesTo
----           -
Secure Rule    Secure!      @RESOURCE.EnableSecure == 1
Main Rule      NotSecure!

PS> $sd = $rules[0].SecurityDescriptor
PS> Format-NtSecurityDescriptor $sd -Type File -SecurityInformation Dacl
<DACL> (Auto Inherit Requested)
- Type      : AllowedCallback
- Name      : Everyone
- SID       : S-1-1-0
- Mask      : 0x001F01FF
- Access    : Full Access
- Flags     : None
- Condition: @USER.ad://ext/clearance == "TS/ST3" &&
              @DEVICE.ad://ext/location = "Secure"

```

*Listing 7-30: Displaying the central access policy*

Here, when we run `Get-CentralAccessPolicy` we see two policies, `Secure Room Policy` and `Main Policy`. Each policy has a `CapId` SID and a `Rules` property, which we can expand to see the individual rules. The output table contains the following fields: `Name`, `Description`, and `AppliesTo`, which is a conditional expression used to select whether the rule should be enforced. If the `AppliesTo` field is empty, the rule will always be enforced. The `AppliesTo` field for the `Secure Rule` selects on a resource attribute, which we'll come back to in [Listing 7-32](#).

Let's display the security descriptor for this rule. The DACL contains a single `AllowedCallback` ACE that grants full access to the `Everyone` group if the condition matches. In this case, the clearance user claim must be set to the value `TS/ST3`, and the device claim location must be set to `Secure`.

We'll walk through a basic implementation of the central access policy access check to better understand what the policy is being used for. Add the code in [Listing 7-31](#) to the end of the `Get-PSGrantedAccess` function from [Listing 7-2](#).

```

❶ if (!$success) {
    return Get-AccessResult STATUS_ACCESS_DENIED
}

❷ $capid = $SecurityDescriptor.ScopedPolicyId
if ($null -eq $capid) {
    return Get-AccessResult STATUS_SUCCESS $Context.Privileges $DesiredAccess
}

❸ $policy = Get-CentralAccessPolicy -CapId $capid.Sid
if ($null -eq $policy){
    return Get-AccessResult STATUS_SUCCESS $Context.Privileges $DesiredAccess
}

❹ $effective_access = $DesiredAccess
foreach($rule in $policy.Rules) {
    if ($rule.AppliesTo -ne "") {
        $resource_attrs = $null
        if ($sd.ResourceAttributes.Count -gt 0) {
            $resource_attrs = $sd.ResourceAttributes.ResourceAttribute
        }
        ❺ if (!(Test-NtAceCondition -Token $Token -Condition $rule.AppliesTo
        -ResourceAttribute $resource_attrs)) {
            continue
        }
    }
    $new_sd = Copy-NtSecurityDescriptor $SecurityDescriptor
    ❻ Set-NtSecurityDescriptorDacl $rule.Sd.Dacl

    $Context.SecurityDescriptor = $new_sd
    $Context.RemainingAccess = $DesiredAccess

    ❼ Get-DiscretionaryAccess $Context
    ❽ $effective_access = $effective_access -band (-bnot $Context.RemainingAccess)
}

❾ if (Test-NtAccessMask $effective_access -Empty) {
    return Get-AccessResult STATUS_ACCESS_DENIED
}

❿ return Get-AccessResult STATUS_SUCCESS $Context.Privileges $effective_access

```

*Listing 7-31: The central access policy check*

**Listing 7-31** begins immediately after the discretionary access check. If this check fails, the `$success` variable will be `False`, and we should return `STATUS_ACCESS_DENIED` ❶. To start the process of enforcing a central access policy, we need to query the `ScopedPolicyId` ACE from the SACL ❷. If there is no `ScopedPolicyId` ACE, we can return success. We also return success if there is no central access policy with a `CapId` that matches the ACE's SID ❸.

Within the central access policy check, we first set the effective access to the original `DesiredAccess` ❹. We'll use the effective access to determine how much of the `DesiredAccess` we can grant after processing all the policy rules. Next, we check the `AppliesTo` conditional expression for each rule. If there is no value, the rule applies to all resources and tokens. If there is a conditional expression, we must check it using `Test-NtAceCondition`, passing any resource attributes from the security descriptor ❺. If the test doesn't pass, the check should skip to the next rule.

We build a new security descriptor using the owner, group, and SACL from the original security descriptor but the DACL from the rule's security descriptor ❻. If the rule applies, we do another discretionary access check for the `DesiredAccess` ❼. After this check, we remove any bits that we weren't granted from the `effective_access` variable ❽.

Once we've checked all the applicable rules, we test whether the effective access is empty. If it is, the central access policy has not granted the token any access, so we return `STATUS_ACCESS_DENIED` ❾. Otherwise, we return success, but we return only the remaining effective access that grants less access than the result of the first access check ❿.

While most central access policies are designed to check files, we can modify any resource type to enforce a policy. To enable it for another resource, we need to do two things: set a scoped policy ID ACE with the SID of the policy to enable, and add any resource attribute ACEs to match the `AppliesTo` condition, if there is one. We perform these tasks in **Listing 7-32**.

```
PS> $sd = New-NtSecurityDescriptor
❶ PS> $attr = New-NtSecurityAttribute "EnableSecure" -LongValue 1
❷ PS> Add-NtSecurityDescriptorAce $sd -Type ResourceAttribute -Sid "WD"
```

```

-SecurityAttribute $attr -Flags ObjectInherit, ContainerInherit
PS> $scapid = "S-1-17-3260955821-1180564752-1365479606-2616254494"
❸ PS> Add-NtSecurityDescriptorAce $sd -Type ScopedPolicyId -Sid $scapid
-Flags ObjectInherit, ContainerInherit
PS> Format-NtSecurityDescriptor $sd -SecurityInformation Attribute, Scope
Type: Generic
Control: SacIPresent
<Resource Attributes>
- Type   : ResourceAttribute
- Name   : Everyone
- SID    : S-1-1-0
- Mask   : 0x00000000
- Access : Full Access
- Flags  : ObjectInherit, ContainerInherit
- Attribute: "EnableSecure",TI,0x0,1

<Scoped Policy ID>
- Type   : ScopedPolicyId
- Name   : S-1-17-3260955821-1180564752-1365479606-2616254494
- SID    : S-1-17-3260955821-1180564752-1365479606-2616254494
- Mask   : 0x00000000
- Access : Full Access
- Flags  : ObjectInherit, ContainerInherit

❹ PS> Enable-NtTokenPrivilege SeSecurityPrivilege
❺ PS> Set-Win32SecurityDescriptor $sd MACHINE\SOFTWARE\PROTECTED
-Type RegistryKey -SecurityInformation Scope, Attribute

```

*Listing 7-32: Enabling the Secure Room Policy for a registry key*

The first thing we need to do is add a resource attribute ACE to satisfy the AppliesTo condition for the Secure Rule. We create a security attribute object with the name `EnableSecure` and a single `Int64` value of 1 ❶. We add this security attribute to an ACE of type `ResourceAttribute` in the security descriptor's SACL ❷. We then need to set the SID of the central access policy, which we can get from the output of the `Get-CentralAccessPolicy` command in a `ScopedPolicyId` ACE ❸. We can format the security descriptor to check that the ACEs are correct.

We now set the two ACEs to the resource. In this case, the resource we'll pick is a registry key ❺. Note that you must have previously created this

registry key for the operation to succeed. The `SecurityInformation` parameter must be set to `Scope` and `Attribute`. As we observed in [Chapter 5](#), to set the `ScopedPolicyId` ACE, we need `AccessSystemSecurity` access, which means we need to first enable `SeSecurityPrivilege` ④.

If you access the registry key, you should find the policy to be enforced. Note that because the central access policy is configured for use with filesystems, the access mask in the security descriptor might not work correctly with other resources, such as registry keys. You could manually configure the attributes in Active Directory if you really wanted to support this behavior.

One final thing to mention is that central access policy rules support specifying a staging security descriptor as well as the normal security descriptor. We can use this staging security descriptor to test an upcoming security change before deploying it widely. The staging security descriptor is checked in the same way as the normal security descriptor, except the result of the check is used only to compare against the real granted access, and an audit log is generated if the two access masks differ.

## Worked Examples

Let's finish with some worked examples using the commands you've learned about in this chapter.

### *Using the `Get-PSGrantedAccess` Command*

Throughout this chapter, we've built our own implementation of the access check process: the `Get-PSGrantedAccess` command. In this section, we'll explore the use of this command. You can retrieve the module containing it from the `chapter_7_access_check_impl.psm1` file included with the online additional materials for this book.

Because `Get-PSGrantedAccess` is a simple implementation of the access check, it's missing some features, such as support for calculating maximum access. However, it can still help you understand the access check process. You can, for example, use a PowerShell debugger in the PowerShell Integrated Scripting Environment (ISE) or Visual Studio Code

to step through the access check and see how it functions based on different input.

Run the commands in [Listing 7-33](#) as a non-administrator split-token user.

```
❶ PS> Import-Module ".\chapter_7_access_check_impl.psm1"
❷ PS> $sd = New-NtSecurityDescriptor "O:SYG:SYD:(A;;GR;;;WD)"
    -Type File -MapGeneric
PS> $type = Get-NtType File
PS> $desired_access = Get-NtAccessMask -FileAccess GenericRead
    -MapGenericRights
❸ PS> Get-PSGrantedAccess -SecurityDescriptor $sd
    -GenericMapping $type.GenericMapping -DesiredAccess $desired_access
Status                Privileges                GrantedAccess
-----
STATUS_SUCCESS        {}                        1179785

❹ PS> $desired_access = Get-NtAccessMask -FileAccess WriteOwner
PS> Get-PSGrantedAccess -SecurityDescriptor $sd
    -GenericMapping $type.GenericMapping -DesiredAccess $desired_access
Status                Privileges                GrantedAccess
-----
STATUS_ACCESS_DENIED {}                        0

❺
❻ PS> $token = Get-NtToken -Linked
❼ PS> Enable-NtTokenPrivilege -Token $token SeTakeOwnershipPrivilege
PS> Get-PSGrantedAccess -Token $token -SecurityDescriptor $sd
    -GenericMapping $type.GenericMapping -DesiredAccess $desired_access
Status                Privileges                GrantedAccess
-----
STATUS_SUCCESS        {SeTakeOwnershipPrivilege} 524288
```

*Listing 7-33: Using the Get-PSGrantedAccess command*

First, we import the module containing the Get-PSGrantedAccess command

❶. The import assumes the module file is saved in your current directory; if it's not, modify the path as appropriate. We then build a restrictive security descriptor, granting read access to the *Everyone* group and nobody else ❷.



Next, we call `Get-PSGrantedAccess`, requesting `GenericRead` access along with the `File` object type's generic mapping ❸. We don't specify a `Token` parameter, which means the check will use the caller's effective token. The command returns `STATUS_SUCCESS`, and the granted access matches the desired access we originally passed to it.

Then we change the desired access to `WriteOwner` access only ❹. Based on the restrictive security descriptor, only the owner of the security descriptor, which was set to the `SYSTEM` user, should be granted this access. When we rerun the access check, we get `STATUS_ACCESS_DENIED` and no granted access ❺.

To show how we can bypass these restrictions, we query for the caller's linked token ❻. As described in [Chapter 4](#), UAC uses the linked token to expose the full administrator token. This command won't work unless you're running the script as a split-token administrator. However, we can enable the `SeTakeOwnershipPrivilege` privilege on the linked token ❼, which should bypass the owner check for `WriteOwner`. The access check should now return `STATUS_SUCCESS` and grant the desired access ❽. The `Privileges` column shows that `SeTakeOwnershipPrivilege` was used to grant the access right.

As mentioned, it's worth running this script in a debugger and stepping into `Get-PSGrantedAccess` to follow along with the access check process so that you understand it better. I also recommend trying different combinations of values in the security descriptor.

### ***Calculating Granted Access for Resources***

If you really need to know the granted access of a resource, you're better off using the `Get-NtGrantedAccess` command over the PowerShell implementation we've developed. Let's see how we can use this command to get the granted access for a list of resources. In [Listing 7-34](#), we'll take the script we used in [Chapter 6](#) to find the owners of objects and calculate the full granted access.

```

PS> function Get-NameAndGrantedAccess {
    [CmdletBinding()]
    param(
        [parameter(Mandatory, ValueFromPipeline)]
        $Entry,
        [parameter(Mandatory)]
        $Root
    )

    PROCESS {
        $sd = Get-NtSecurityDescriptor -Path $Entry.Name -Root $Root
        -TypeName $Entry.NtTypeName -ErrorAction SilentlyContinue
        if ($null -ne $sd) {
            ❶ $granted_access = Get-NtGrantedAccess -SecurityDescriptor $sd
            if (!(Test-NtAccessMask $granted_access -Empty)) {
                $props = @{
                    Name = $Entry.Name;
                    NtTypeName = $Entry.NtTypeName
                    GrantedAccess = $granted_access
                }

                New-Object -TypeName PSObject -Prop $props
            }
        }
    }
}

PS> Use-NtObject($dir = Get-NtDirectory \BaseNamedObjects) {
    Get-NtDirectoryEntry $dir | Get-NameAndGrantedAccess -Root $dir
}

```

| Name                               | NtTypeName | GrantedAccess                |
|------------------------------------|------------|------------------------------|
| ----                               | -----      | -----                        |
| SM0:8924:120:WilError_03_p0        | Semaphore  | QueryState, ModifyState, ... |
| CLR_PerfMon_DoneEnumEvent          | Event      | QueryState, ModifyState, ... |
| msys-2.0S5-1888ae32e00d56aa        | Directory  | Query, Traverse, ...         |
| SyncRootManagerRegistryUpdateEvent | Event      | QueryState, ModifyState, ... |
| --snip--                           |            |                              |

*Listing 7-34: Enumerating objects and getting their granted access*

In this modified version of the script created in [Listing 6-37](#), instead of merely checking the owner SID, we call `Get-NtGrantedAccess` with the secu-

urity descriptor ❶. This should retrieve the granted access for the caller. Another strategy would have been to check the granted access for any impersonation token at the Identification level with `query` access on the handle, then pass it as the `Token` parameter. In the next chapter, we'll explore an easier way to do large-scale access checking without having to write your own scripts.

## Wrapping Up

In this chapter, we detailed the implementation of the access check process in Windows at length. This included describing the operating system's mandatory access checks, token owner and privilege checks, and discretionary access checks. We also built our own implementation of the access check process to enable you to better understand it.

Next, we covered how the two types of sandboxing tokens (restricted and lowbox) affect the access check process to restrict resource access. Finally, we discussed object type checking and central access policies, important features of enterprise security for Windows.